DAY 26 / 90

AWS Week — Day 4: RDS & Databases

Managed PostgreSQL • Private access • Backups and restore

Goal: Understand managed databases and, if your account and budget allow, build a private Amazon RDS PostgreSQL database reachable from EC2 only. Create a snapshot, restore it to a new instance, verify the data, then clean up.

Scope rule: Do not repeat Day 22 AWS overview/account setup, Day 23 IAM, Day 24 S3/CloudFront, or Day 25 EC2 setup. Reuse your earlier EC2 and IAM setup if available; otherwise use the concept-only path.

COST WARNING: RDS can incur charges. Check current pricing before creating resources. Do not create a NAT Gateway or Multi-AZ database for this practice unless you have a specific need and have checked pricing. If unsure, complete the design, SQL and interview sections without launching RDS. Budget alerts are not a hard spending cap.

1. Key concepts

Concept	Simple explanation
RDS	AWS manages database infrastructure, patching, backups, monitoring and recovery options. You manage schema, queries, data and access.
DB subnet group	A collection of subnets in multiple Availability Zones that tells RDS where it may place a database.
DB security group	Controls network access. For PostgreSQL, allow TCP 5432 only from the EC2 app security group.
Automated backups	Support point-in-time recovery within the configured retention period (1–35 days).
Manual snapshot	A saved backup retained until deleted. Restoring it creates a new DB instance.
Multi-AZ	High availability with a standby and automatic failover; classic Multi-AZ standby does not serve read traffic.
Read replica	An asynchronous copy used mainly to scale reads; replication can lag.
DynamoDB	Managed NoSQL key-value/document database for high-scale, predictable access patterns; no relational joins.

2. Architecture and security

Access EC2 through your previously configured method (prefer Session Manager where available). EC2 connects to the private RDS database on TCP 5432. The internet must not connect directly to RDS.

```
You -- SSM/SSH --> EC2 (app-sg) ---- TCP 5432 ----> RDS PostgreSQL (db-sg)
Internet ------------------- X ----------------------> RDS
RDS Publicly accessible = No
db-sg inbound: PostgreSQL / TCP 5432 / source = EC2 app-sg
```

Never open database port 5432 to 0.0.0.0/0. Set the inbound source to the EC2 security group, not a public IP address.

3. Task A — Plan first

• Identify the VPC and EC2 instance from Day 25; record the EC2 security group ID and VPC ID.

• Check that suitable subnets exist in at least two Availability Zones for the DB subnet group. If not, do not create a database just for this lab.

• Plan a DB security group (e.g., db-sg) with one inbound rule: TCP 5432 from the EC2 app security group only.

• Choose a small supported development configuration only after checking current pricing. Keep public access disabled.

• Plan secure credentials. Prefer Secrets Manager or IAM database authentication where supported; never hardcode passwords in scripts, Git or container images.

Record: region, VPC ID, EC2 security group ID, DB subnet group, DB security group, engine, backup retention and the pricing page checked.

4. Task B — Create RDS (optional paid lab)

Proceed only if pricing and permissions are confirmed. You may use the AWS Console. Names below are examples; never put passwords in notes or screenshots.

• Open RDS → Databases → Create database; choose PostgreSQL and a development/test option suitable for your account.

• Choose a unique DB identifier and username. Store the password securely; do not reuse your AWS console password.

• Select the intended VPC and DB subnet group. Set Public access to No.

• Attach db-sg. Allow TCP 5432 only from the EC2 app security group.

• Set automated backup retention to 7 days if supported. Avoid Multi-AZ for this cost-conscious lab unless justified and priced.

• Create the database and wait until status is Available; provisioning may take several minutes.

If you cannot safely create RDS, document the intended settings and continue with the SQL, service comparison and interview sections.

5. Task C — Connect from EC2 and test SQL

Use a PostgreSQL client on EC2. Connect using the RDS endpoint and credentials retrieved securely. Avoid putting the password in a command saved to shell history.

```
psql "host=<RDS_ENDPOINT> port=5432 dbname=postgres user=<DB_USER> sslmode=require"
```

At the psql prompt, run:

```
CREATE TABLE lab_notes (
id SERIAL PRIMARY KEY,
note TEXT NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO lab_notes (note)
VALUES ('Day 26 RDS connection test');

SELECT id, note, created_at FROM lab_notes;
```

Verify the row appears. A direct laptop connection should fail/time out because RDS is private and only EC2 is allowed. Do not make RDS public to make the test pass.

6. Task D — Snapshot, restore, verify

• Create a manual snapshot named something like day26-before-restore.

• After it is available, delete the test row or add a second row and confirm the change.

• Restore the snapshot to a new DB instance with a different identifier. Keep it private and attach the correct DB security group.

• Wait until the restored instance is Available. It will have a new endpoint.

• Connect from EC2 to the restored endpoint and query lab_notes. Confirm the original row from the snapshot is present.

• Record the result, then delete both instances and lab snapshots after the evidence is saved.

Remember: Restore creates a new instance; it does not overwrite the original. Snapshots can incur storage charges until deleted.

7. Which database service?

Service	Choose it when…
RDS	You need managed relational SQL, joins and transactions.
Aurora	You need cloud-optimized MySQL/PostgreSQL-compatible relational database capabilities and can justify the cost.

Service	Choose it when…
DynamoDB	You need high-scale key-value/document access with predictable patterns, without relational joins.
ElastiCache	You need in-memory caching for hot data, sessions or rate limiting.
Self-managed DB on EC2	You need OS-level control and accept responsibility for patching, backups and operations.

8. Troubleshooting

Problem	Check
psql hangs / times out	Check VPC, routes, db-sg TCP 5432 from the EC2 security group, and subnet configuration.
Password authentication failed	Check username and retrieve the current password securely from the configured secret.
RDS still creating	Wait for Available; provisioning may take 5–10 minutes or longer.
Restore has a different endpoint	Expected. Update the connection command or application configuration.
EC2 cannot read Secrets Manager	Check the EC2 role has least-privilege permission and network access to the service.
Laptop cannot connect	Expected for a private DB. Do not make it public; connect through the approved EC2 path.

9. Cleanup — do not skip

• Delete the original RDS instance and the restored instance after testing. Carefully review final-snapshot options so you do not create extra snapshots unintentionally.

• Delete manual snapshots created for this lab once proof is saved and they are no longer needed.

• Check RDS → Databases and Snapshots for remaining resources.

• Remove the lab DB security group and subnet group only if no remaining resources use them.

• Do not delete shared VPC, EC2, IAM roles or security groups from earlier days unless certain they are lab-only and unused.

• Review billing/costs later; deleting resources may not immediately remove every finalized charge.

RDS instances, storage, snapshots/backups, data transfer and related networking may cost money. Do not leave the database running overnight.

10. Deliverable

Create a folder named **day4-rds/** in your local notes/repository containing:

• create-db.sh or step-by-step creation notes (no secrets/passwords).

• SQL file with table creation, insert and select statements.

• A simple EC2 → private RDS diagram with the TCP 5432 security-group rule.

• A short cost note explaining what you checked and which resources you deleted.

• Short answers to the interview questions below.

Take only 3–5 useful screenshots: (1) RDS configuration showing Public access = No, (2) db-sg inbound rule showing EC2 security group as source, (3) successful SQL query from EC2, (4) restore verification showing the original row, (5) cleanup proof showing no lab DBs/snapshots remain. Hide account IDs and never show secret values.

11. Interview questions — short answers

Q. What does RDS manage?
AWS manages much of the infrastructure, patching, backups, monitoring and recovery features; you manage schema, queries and access.

Q. Multi-AZ vs read replica?
Multi-AZ is mainly for high availability and automatic failover. A read replica is asynchronous and scales read traffic; it may lag.

Q. How do you keep RDS private?
Use private subnets where appropriate, set Publicly accessible to No, and allow the DB port only from the application security group.

Q. How do RDS backups work?
Automated backups support point-in-time recovery within retention. Manual snapshots remain until deleted. Restore creates a new instance.

Q. RDS vs Aurora?
Both support relational workloads. Aurora is cloud-optimized and may offer higher performance/faster failover, but can cost more.

Q. When choose DynamoDB over RDS?
For high-scale key-value/document access with known patterns and no relational joins; RDS for SQL, joins and transactions.

Q. How should DB credentials be managed?
Use Secrets Manager with suitable rotation or IAM database authentication where supported. Never hardcode credentials.

Q. How can you test a database change safely?
Take a snapshot, restore a separate test instance, verify the change, then plan a controlled rollout.

12. Completion checklist

• I can explain RDS, DB subnet groups and DB security groups.

• I understand why the database should not be publicly accessible.

• I can explain Multi-AZ versus read replicas.

• I can describe snapshot restore and why the restored instance has a new endpoint.

• I documented the SQL test and restore result, or completed the concept-only path safely.

• I checked cleanup and removed the lab RDS instances/snapshots I created.

Learn → Build → Break → Fix → Explain